Boletín U11 — Intermedio

📝 Boletín U11 — Intermedio

Usa la colección `peliculas` de la base `cine` del boletín inicial, con documentos `{ titulo, anio, genero, nota }`. Los ejercicios con 🧪 usan db4o y son opcionales.

1. Crea la clase `Pelicula` (título, año, género, nota) y la clase `PeliculaDao` con los métodos `insertar(Pelicula)`, `buscarPorTitulo(String)`, `buscarTodas()`, `actualizarNota(String titulo, double nota)` y `borrar(String titulo)`. Incluye métodos privados para convertir entre `Pelicula` y `Document`. Escribe un `main` que pruebe todos.

2. Añade a `PeliculaDao` un método `buscar(String genero, int anioMinimo, double notaMinima)` que devuelva las películas del género indicado, posteriores a ese año **y** con esa nota mínima, ordenadas por nota descendente y limitadas a 10 resultados. Usa `Filters.and`, `Sorts` y `limit`.

3. Añade un método que devuelva las películas que sean de terror **o** de ciencia ficción **o** que tengan una nota superior a 9. Usa `Filters.or`.

4. Añade un método `subirNotaGenero(String genero, double incremento)` que sume el incremento a la nota de todas las películas de ese género con `updateMany` y `Updates.inc`, y devuelva cuántos documentos se modificaron (`getModifiedCount()`).

5. Diseña y guarda documentos de una colección `libros` con el autor **embebido**:

   ```
   { "titulo": "...", "autor": { "nombre": "...", "pais": "..." },
   "etiquetas": ["...", "..."] }
   ```

   Escribe, en el shell **y** en Java, la consulta que devuelva los libros cuyo autor sea de un país concreto (notación de punto: `autor.pais`) y otra que devuelva los que tengan una etiqueta concreta (MongoDB busca dentro de las listas con una consulta de igualdad normal).

6. Con el *aggregation framework* (`Aggregates.group` y `Accumulators`), obtén cuántas películas hay de cada género y la nota media de cada uno. Muestra el resultado en pantalla.

7. Crea un índice sobre el campo `titulo` con `createIndex(Indexes.ascending("titulo"))`. Después crea un índice **único** y comprueba qué ocurre al intentar insertar dos películas con el mismo título. ¿Qué excepción salta?

8. **Diseño.** Una tienda online tiene clientes, productos y pedidos. Un pedido pertenece a un cliente y contiene varias líneas (producto + cantidad). Propón cómo organizarías las colecciones en MongoDB y qué relaciones **embeberías** y cuáles **referenciarías**. Justifica cada decisión y señala qué consulta se vuelve más cómoda y cuál más incómoda con tu diseño.

9. 💉 Con db4o, crea la clase `Coche` (marca y un `Piloto`). Almacena varios coches con sus pilotos, y con una **consulta nativa** (`Predicate`) recupera los coches cuyo piloto tenga más de 50 puntos.

10. 💉 Con la configuración por defecto, recupera un `Coche`, súmale puntos a su piloto y guarda el coche. Cierra y abre la base: ¿se ha guardado el cambio? Corrige el programa para que se guarde con `cascadeOnUpdate(true)` y explica qué ocurre además con `delete(coche)` sin `cascadeOnDelete(true)`.

11. Compara en una tabla, con tus palabras, estas tres formas de guardar una colección de `Pelicula`: una tabla relacional con JDBC (U10), una BDOO y MongoDB. Considera: esquema, consultas, relaciones entre datos, transacciones y madurez de la tecnología.